

A language model you did not train, and its voice after thirty sentences

Everything is written and tested. Sample from a small pretrained transformer at three temperatures, fine-tune it on a thirty-sentence course corpus, and compare the same prompt before and after.

Guided lab

25 minutes

Hugging Face transformers

CPU works · GPU is quicker

Before you start 1 · load the model 1 · temperature is not... 2 · fine-tune on the course...
3 · before and after,... Reflection If you finish early

Before you start

Open the provided `lab17.ipynb` with the `data/` folder beside it. The first run downloads `distilgpt2` (about 350 MB), so start that cell as the lecture ends.



By the end of this lab you can

- Generate text from a pretrained causal language model.
- Explain temperature as a sampling control, not a knowledge control.
- Fine-tune the model on a small corpus and show the change with a controlled comparison.

STEP 1 · GIVEN

load the model

1

`distilgpt2` has 82 million parameters and runs on a laptop CPU. The course does not ask you to pretrain one; session 27 explains why a frontier model is a different animal.

given

PYTHON

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_NAME = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME)
tokenizer.pad_token = tokenizer.eos_token

def generate(prompt, temperature=0.8, seed=7, max_new_tokens=40):
    torch.manual_seed(seed)
    x = tokenizer(prompt, return_tensors="pt")
    with torch.no_grad():
        y = model.generate(**x, do_sample=True, temperature=temperature,
max_new_tokens=max_new_tokens,
                        pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(y[0], skip_special_tokens=True)

print(f"{sum(p.numel() for p in model.parameters()):,} parameters")
```

TASK 1 · 7 MINUTES

temperature is not knowledge

1

Run the prompt at three temperatures and two seeds. Change only the prompt text if you like.

your turn

PYTHON

```
prompt = "Artificial intelligence students should" # TODO (optional): your own prompt
for temp in [0.2, 0.8, 1.5]:
    for seed in [7, 8]:
        print(f"\n- temperature {temp}, seed {seed}\n{generate(prompt, temperature=temp,
seed=seed)}")
```

Temperature	Variety between seeds	Repetition	Coherence	Anything factually checkable?
0.2				
0.8				
1.5				

In one sentence: why does raising the temperature change *how* the model samples but not *what* it knows?

TASK 2 · 8 MINUTES

fine-tune on the course corpus · given

2

`data/csc4315_finetune_corpus.txt` is thirty sentences, every one beginning *"In CSC 4315,"*. That is deliberately tiny: the aim is to make adaptation **visible**, not to make a good model. The `before` text is generated first, with a fixed prompt, seed and temperature.

given

PYTHON

```
from torch.utils.data import DataLoader, TensorDataset

PROBE = dict(prompt="In CSC 4315,", temperature=0.8, seed=11, max_new_tokens=30)
before = generate(**PROBE)

corpus = open("data/csc4315_finetune_corpus.txt", encoding="utf-8").read()
ids = tokenizer(corpus, return_tensors="pt")["input_ids"][0]
block = 32
chunks = torch.stack([ids[i:i + block] for i in range(0, len(ids) - block, block // 2)])
loader = DataLoader(TensorDataset(chunks), batch_size=4, shuffle=True)

model.train()
opt = torch.optim.AdamW(model.parameters(), lr=5e-5)
for epoch in range(4):
    for (batch,) in loader:
        out = model(input_ids=batch, labels=batch)
        out.loss.backward(); opt.step(); opt.zero_grad()
        print(f"epoch {epoch + 1}: loss {out.loss.item():.3f}")
model.eval()
after = generate(**PROBE)
```

before and after, controlled

3

in the notebook

PYTHON

```
print("BEFORE:\n", before)
print("\nAFTER:\n", after)
```

Same prompt, seed and temperature — so the only thing that changed is the parameters.

1. Name **two** changes in the after-text that you can trace to specific sentences in the corpus.
2. Give **one** reason this does not show that the model understands the course.

Your answer:

Reflection



Write it in the notebook, in your own words

Session 23 fine-tuned an image network; today you fine-tuned a text network. In two sentences: what was the same move, and what was different about the data?

Your answer:

If you finish early

A. Fine-tune for 12 epochs instead of 4 and regenerate. At what point does the model start repeating corpus sentences word for word — and which session's idea names what you are seeing?

your turn

PYTHON

```
# TODO (optional extension A)
```